SQL Server Enterprise Reliability and Administration

Complete study notes • backups • recovery • HA/DR • Availability Groups • replication • integrity • maintenance • interview revision

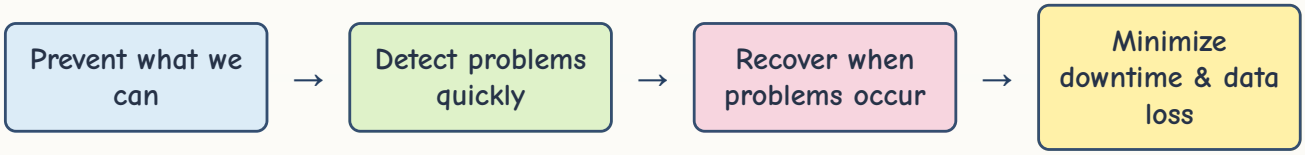
Enterprise database reliability means:
Keeping the database available, protecting data from loss, recovering correctly after failures, and continuously checking that the database remains healthy.

A production SQL Server system can fail because of hardware failure, disk or storage failure, server crash, accidental `DELETE`, accidental `DROP TABLE`, a bad application deployment, database corruption, network failure, an entire data-center failure, or human error.

Enterprise reliability is therefore not one feature. It is a combination of:

Backups + Recovery strategy + High availability + Disaster recovery + Failover + Replication where appropriate + Integrity checking + Regular restore testing + Monitoring and maintenance

For `FundraisingDb`, imagine that it contains years of Constituents, Gifts, GiftAllocations, Campaigns, Funds, Interactions, and Tasks. Losing this data would be a serious business problem.



✦ 1. Reliability, High Availability, and Disaster Recovery

These terms are related, but they are not identical.

Reliability

Reliability means the database system behaves correctly and can recover from failures. It includes backups, integrity checks, monitoring, recovery planning, redundancy, and maintenance.

High Availability - HA

High Availability means reducing downtime when a server or database service becomes unavailable. The key question is: *How quickly can another working system continue serving users?*

Examples include Always On Availability Groups, Failover Cluster Instances, and other redundant server designs.

Disaster Recovery - DR

Disaster Recovery means restoring service after a major failure that can affect the original environment. Examples include an entire server being lost, a data center becoming unavailable, storage destruction, ransomware, or a regional outage.

DR commonly requires backups, off-site copies, secondary systems, and documented recovery procedures.

Backup vs High Availability

Backup ≠ High Availability.
Backup protects historical data and allows restoration. **HA** reduces service interruption by providing another available system.

An Availability Group secondary is **not a replacement for backups**. Secondary databases are redundant copies, not historical backup sets; regular database and transaction-log backups are still required.

Example - accidental DELETE

```sql
DELETE
FROM fundraising.Gift;
```

If the delete is committed, it can be replicated as a legitimate change:

EXAMPLE / FLOW

```
Primary   -> Gifts deleted
Secondary -> Gifts deleted
```

High availability did not protect against the logical mistake. A backup/recovery strategy can allow recovery to a time just before the `DELETE`.

HA + Backups are both needed

✦ 2. Two Important Business Concepts - RPO and RTO

RPO - Recovery Point Objective

RPO describes how much data loss the business is willing to accept. The question is: *If the database fails, how far back can we afford to recover?*

Example: `RPO = 15 minutes` means losing more than approximately 15 minutes of committed business data is unacceptable.

RPO strongly influences transaction-log backup frequency, secondary-replica design, and disaster-recovery architecture.

RTO - Recovery Time Objective

RTO describes how long the business can tolerate the system being unavailable. Example: `RTO = 30 minutes` means the database service should be recovered within approximately 30 minutes.

RTO affects restore strategy, backup size, differential frequency, HA architecture, automation, and failover capability.

RPO	RTO
How much DATA can we lose?	How much TIME can we be down?

✦ 3. What Is a Backup?

A **backup is a recoverable copy of database data and/or transaction-log information that can later be restored.** SQL Server backups protect against media failure, hardware failure, accidental data deletion, server loss, and disaster.

A reliable enterprise strategy also includes tested restore procedures and safely stored copies away from the primary database storage.

Important backup types

Focus first on:

- Full Backup
- Differential Backup
- Transaction Log Backup

SQL Server also supports file backups, filegroup backups, partial backups, and copy-only backups, but the first three form the core strategy for many application databases.

✦ 4. Full Backup

A **full database backup contains the entire database plus enough transaction-log information to make the restored database transactionally consistent.** It represents the whole database as of the backup completion point.



Create a full backup

```sql
BACKUP DATABASE FundraisingDb
TO DISK = 'D:\SQLBackups\FundraisingDb_Full.bak'
WITH
    CHECKSUM,
    STATS = 10;
```

What does CHECKSUM do?

`WITH CHECKSUM` asks SQL Server to perform backup checksum validation while producing the backup. This improves the ability to detect certain corruption problems during backup and later verification or restore.

Why is a full backup important?

It provides the starting foundation for many recovery sequences. A common example is a Sunday full backup followed by differential backups and transaction-log backups during the week.

Advantages

- Simple to understand.
- Contains the entire database.
- Straightforward restore starting point.
- Good baseline for recovery.

Disadvantages

For a large database it takes longer, uses more backup storage, requires more I/O, and may take longer to restore. As databases grow, differential backups are often added to reduce backup and restore work.

Does a full backup delete the transaction log?

No. A common misunderstanding is that a full backup automatically truncates the transaction log. In the FULL recovery model, transaction-log backups are what normally allow inactive portions of the log to become reusable after the log records are no longer needed for other reasons.

✦ 5. Differential Backup

A **differential backup contains data extents changed since the most recent full backup that serves as its differential base.**

EXAMPLE / FLOW

```
Sunday FULL
   |
   +-- changes Monday
   +-- changes Tuesday
   +-- changes Wednesday
```

A Wednesday differential contains **all changes since Sunday full**, not only Wednesday's changes.

Important differential rule

EXAMPLE / FLOW

```
Monday Differential    -> changes Sunday to Monday
Tuesday Differential   -> changes Sunday to Tuesday
Wednesday Differential -> changes Sunday to Wednesday
```

Differentials are cumulative relative to the base full backup. They do not normally depend on the previous differential backup.

Create a differential backup

SQL

```sql
BACKUP DATABASE FundraisingDb
TO DISK = 'D:\SQLBackups\FundraisingDb_Diff.bak'
WITH
    DIFFERENTIAL,
    CHECKSUM,
    STATS = 10;
```

Why use differential backups?

If `FundraisingDb` is 2 TB but only 50 GB changes each day, a full 2-TB backup every few hours may be unnecessary. A practical design could use a weekly full, daily differential, and frequent log backups.

Restore benefit

If failure occurs Friday:

EXAMPLE / FLOW

```
Without differential:
Restore Sunday full + many log backups

With Thursday differential:
Restore Sunday full + Thursday differential + only logs after that differential
```

Full vs Differential

Full	Differential
Entire database	Changes since base full
Larger	Usually smaller
Independent restore base	Requires base full
Often longer to create	Often faster
Basis for differential	Cumulative from full

Differential growth

Differentials often grow as more pages change after the base full backup. Monday may be small, Tuesday larger, Wednesday larger, and so on. When a differential approaches a significant part of the full backup size, taking a new full backup may be appropriate.

✦ 6. Transaction Log Backup

A **transaction log backup saves transaction-log records that have not already been backed up in the log-backup chain.** Log backups are central to point-in-time recovery, low data-loss RPOs, and transaction-log reuse.

They apply to the `FULL` and `BULK_LOGGED` recovery models, not to `SIMPLE` in the normal log-backup-chain sense.

Fundraising example

EXAMPLE / FLOW

```
10:00 AM  Gift 10000 inserted
10:02 AM  Gift 10001 inserted
10:06 AM  GiftAllocation created
10:09 AM  Task updated
10:10 AM  Log backup captures relevant unbacked log records
```

Create a log backup

SQL

```sql
BACKUP LOG FundraisingDb
TO DISK = 'D:\SQLBackups\FundraisingDb_Log_1010.trn'
WITH
    CHECKSUM,
    STATS = 10;
```

Why use log backups?

If the full backup is at midnight, a differential at noon, and the database fails at 3:17 PM, log backups every 10 minutes may allow recovery close to 3:17 PM instead of losing everything since noon.

Example backup schedule

EXAMPLE / FLOW

```
Sunday 12 AM   FULL
Every night    DIFFERENTIAL
Every 10 min   TRANSACTION LOG
```

This is a learning example only. Real schedules depend on RPO, RTO, database size, change rate, restore testing, and storage.

Transaction log backup chain



The required log chain must remain intact. Missing an essential log backup can prevent recovery beyond that gap.

✦ 7. Recovery Models

A **recovery model is a database setting that controls transaction-log maintenance and determines which backup/recovery scenarios are available.** SQL Server has three recovery models:

EXAMPLE / FLOW

```
SIMPLE
FULL
BULK_LOGGED
```

For most application-learning scenarios, the main focus is `SIMPLE` and `FULL`.

Check the recovery model

SQL

```sql
SELECT
    name,
    recovery_model_desc
FROM sys.databases
WHERE name = 'FundraisingDb';
```

✦ 8. SIMPLE Recovery Model

Under **SIMPLE**, SQL Server automatically reuses inactive transaction-log space after checkpoints when that log is no longer needed for recovery or other engine requirements. You do not maintain a normal transaction-log backup chain for point-in-time recovery.

Set SIMPLE recovery

SQL

```sql
ALTER DATABASE FundraisingDb
SET RECOVERY SIMPLE;
```

What backups can you use?

Typically full and differential backups, but not regular transaction-log backups used for point-in-time restore.

Recovery example

EXAMPLE / FLOW

```
Full backup         Sunday
Differential        Wednesday 12 PM
Accidental DELETE   Wednesday 4 PM
```

Under SIMPLE, you cannot normally use log backups to recover to 3:59 PM. You might only recover to the Wednesday 12 PM differential, potentially losing about four hours of work.

Advantages

- Simpler log administration.
- No transaction-log backup schedule is needed.
- Suitable when some recent data loss is acceptable.

Disadvantages

- No normal point-in-time log recovery.
- Potentially greater work or data loss.
- Often inappropriate for mission-critical OLTP databases.

When SIMPLE can be reasonable

Development, test, rebuildable reporting databases, data-warehouse staging databases, or databases where recent data loss is acceptable.

When SIMPLE is usually a poor choice

A mission-critical production `FundraisingDb` may require a low RPO, point-in-time restore, and frequent donation processing. In that situation FULL recovery is often more appropriate. The final choice must be based on business RPO/RTO requirements.

✦ 9. FULL Recovery Model

Under **FULL**, SQL Server keeps log records required for the backup chain until they can be backed up and are otherwise eligible for reuse. It supports transaction-log backups, point-in-time recovery, and low data-loss recovery strategies.

Set FULL recovery

SQL

```
ALTER DATABASE FundraisingDb
SET RECOVERY FULL;
```

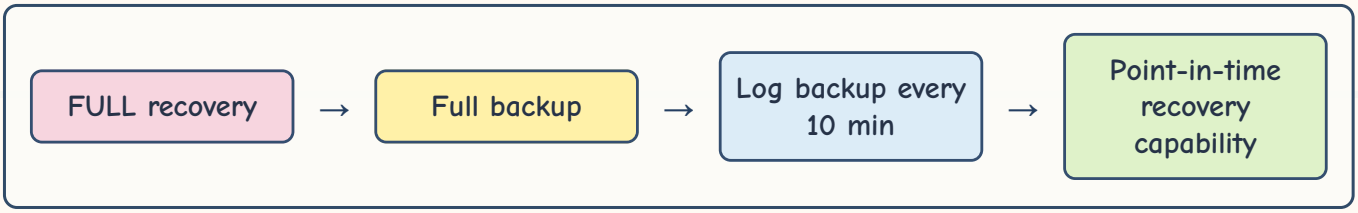
After changing from SIMPLE to FULL, establish the full-recovery log chain with a data backup.

```sql
BACKUP DATABASE FundraisingDb
TO DISK = 'D:\SQLBackups\FundraisingDb_Full.bak'
WITH CHECKSUM;
```

Then schedule transaction-log backups.

FULL recovery requires log backups

A common mistake is setting `RECOVERY FULL` but never running `BACKUP LOG`. That is poor administration. The transaction log can continue growing because SQL Server cannot mark portions reusable merely because FULL recovery was selected.



Advantages

- Point-in-time recovery.
- Transaction-log backups.
- Low RPO is possible.
- Strong production recovery options.

Disadvantages

- Requires regular log backups.
- More administration and backup files.
- Restore sequences are more complex.
- Log growth can become serious if backups are mismanaged.

SIMPLE vs FULL

SIMPLE	FULL
Simpler	More administrative work
No normal log backup chain	Transaction-log backups
No normal point-in-time recovery	Point-in-time recovery supported
Larger potential data loss	Can achieve low RPO
Often noncritical systems	Often important production OLTP

✦ 10. BULK_LOGGED Recovery Model

`BULK_LOGGED` is similar to FULL but can reduce logging for certain bulk operations. It is typically used temporarily during appropriate large bulk operations.

Important limitation: if a log backup contains minimally logged bulk operations, you cannot restore to an arbitrary point inside that log backup; recovery may have to go to the end of the log backup.

For normal learning:

EXAMPLE / FLOW

```
Most application databases -> SIMPLE or FULL
BULK_LOGGED                -> specialized bulk-load scenario
```

✦ 11. Backup Is Only Useful If You Can Restore It

Successful BACKUP command ≠ Guaranteed successful disaster recovery.

A reliable backup strategy requires backup creation, backup monitoring, backup validation, restore testing, off-site or separate storage, and a documented restore procedure.

✦ 12. Backup Validation with RESTORE VERIFYONLY

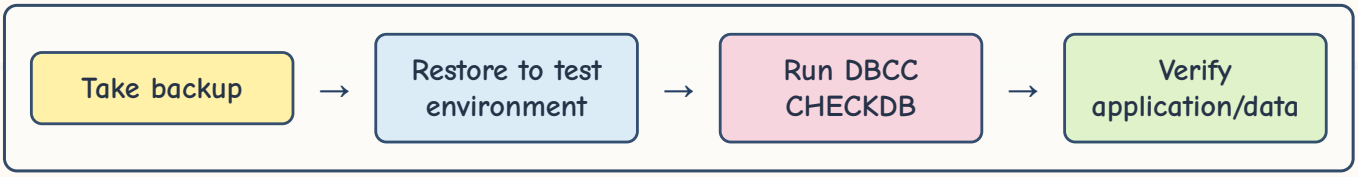
SQL

```
RESTORE VERIFYONLY
FROM DISK = 'D:\SQLBackups\FundraisingDb_Full.bak';
```

`RESTORE VERIFYONLY` checks that the backup set is complete and readable without restoring it. However, it does **not** perform a complete structural validation of the database contained inside the backup.

VERIFYONLY is useful, but VERIFYONLY ≠ complete restore test

Better reliability practice



That provides much stronger confidence.

✦ 13. Restore vs Recovery

Restore

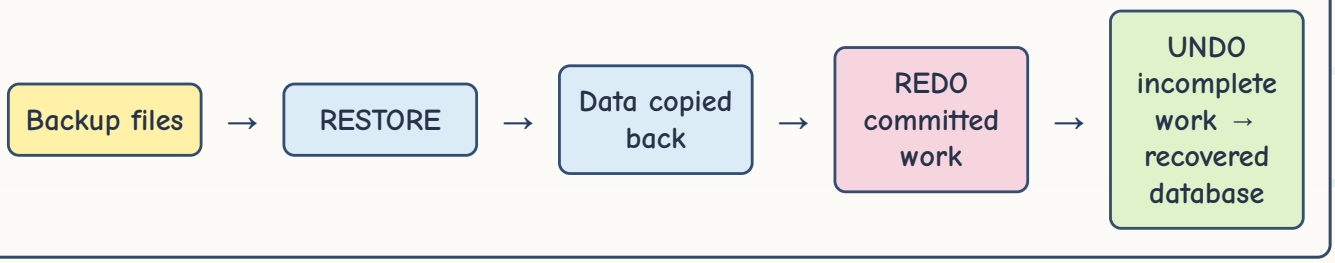
Restore means copying data/log pages from backup media back into database files. Example:

`RESTORE DATABASE ...`

Recovery

Recovery means bringing restored database pages to a transactionally consistent state by applying logged changes and rolling back incomplete transactions.

Simplified restore process



✦ 14. WITH NORECOVERY

When multiple backups must be restored, keep the database in `RESTORING` state until the final backup is applied.

Restore full with NORECOVERY

```sql
RESTORE DATABASE FundraisingDb
FROM DISK = 'D:\SQLBackups\FundraisingDb_Full.bak'
WITH NORECOVERY;
```

Meaning: restore this backup, but do not bring the database online yet because more backups are coming.

Differential restore

```sql
RESTORE DATABASE FundraisingDb
FROM DISK = 'D:\SQLBackups\FundraisingDb_Diff.bak'
WITH NORECOVERY;
```

Log restore

```sql
RESTORE LOG FundraisingDb
FROM DISK = 'D:\SQLBackups\FundraisingDb_Log_1.trn'
WITH NORECOVERY;
```

Repeat for the required log backups.

Final recovery

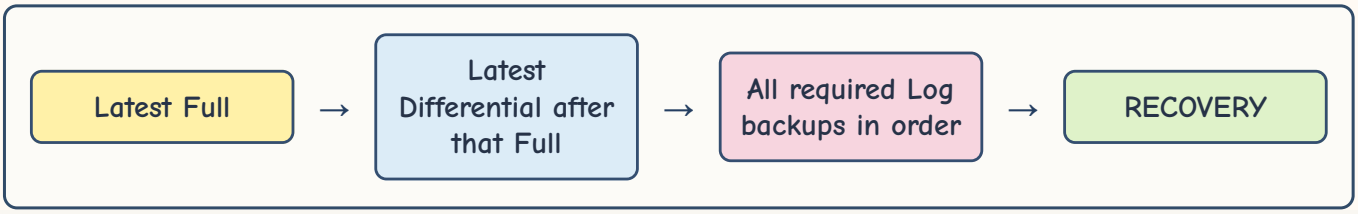
```sql
RESTORE LOG FundraisingDb
FROM DISK = 'D:\SQLBackups\FundraisingDb_Log_Final.trn'
WITH RECOVERY;
```

SQL Server finishes recovery and brings the database online.

NORECOVERY	RECOVERY
More backups are coming.	Restore sequence is finished.

✦ 15. Restore Sequence

Under FULL recovery, a common sequence is:



Important differential restore rule

You do **not** restore Monday, Tuesday, and Wednesday differentials one after another. If Wednesday is the latest differential based on the full backup, restore the full, then Wednesday differential, then the logs required after that differential.

✦ 16. Point-in-Time Recovery

Point-in-time recovery restores a database to a specific time before an unwanted event occurred. This is one of the most important reasons to use FULL recovery plus transaction-log backups.

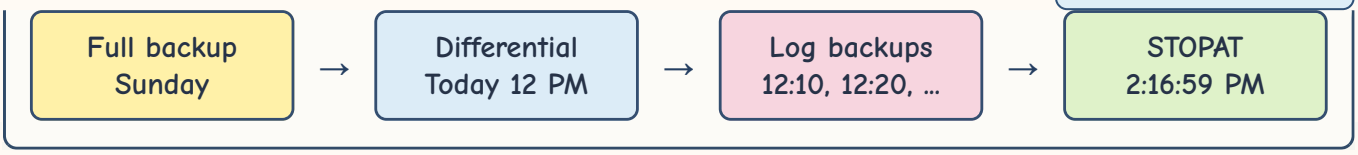
Fundraising example

EXAMPLE / FLOW

```
2:00 PM   Database healthy
2:14 PM   Gift 10500 inserted
2:16 PM   Gift 10501 inserted
2:17 PM   Administrator accidentally runs DELETE FROM fundraising.Gift;
2:18 PM   Problem discovered
```

Goal: recover to `2:16:59 PM`, just before the accidental delete.

Point-in-time flow



Example STOPAT

```sql
RESTORE LOG FundraisingDb
FROM DISK = 'D:\SQLBackups\FundraisingDb_Log.trn'
WITH
    STOPAT = '2026-09-22T14:16:59',
    RECOVERY;
```

A complete real restore sequence first restores the required full, differential, and log backups.

Tail-log backup

If the database has failed but the transaction-log file is still accessible, you may be able to create a **tail-log backup** to capture log records that have not yet been backed up. This can minimize data loss immediately before beginning the restore sequence.

✦ 17. Example Production Backup Strategy

Suppose the business requirements are:

```
RPO <= 15 minutes
RTO <= 1 hour
```

A possible starting strategy:

```
Sunday 12:00 AM   Full backup
Every day 12:00 AM Differential
Every 10 minutes   Transaction log backup
```

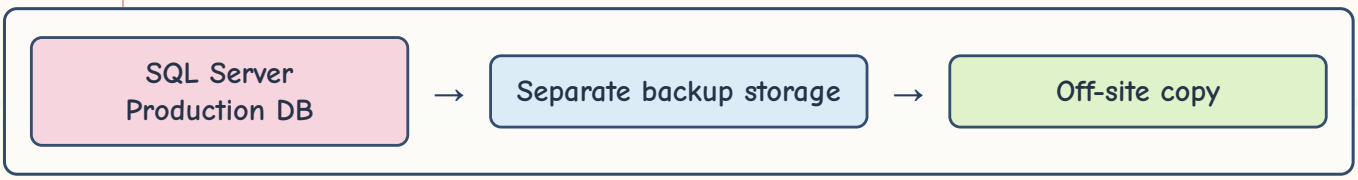
This is a learning example, not a universal production schedule. Actual strategy depends on database size, backup duration, restore duration, workload, storage, RPO, and RTO.

✦ 18. Why Backup Storage Location Matters

Poor design:

```
Database files: Drive D
Backups:        same physical Drive D
```

If the disk fails, both the database and backups can be lost. Backups should be stored on storage physically separate from the database storage, with off-site copies for disaster scenarios.



✦ 19. What Is High Availability?

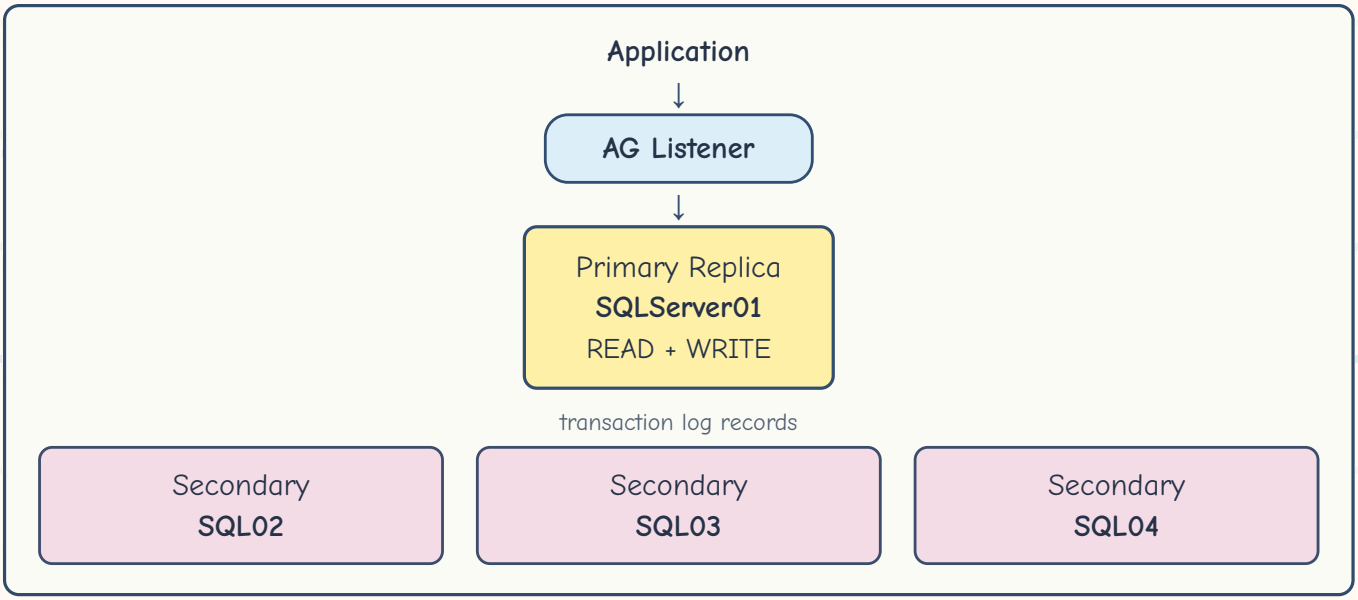
High Availability keeps database services accessible when part of the infrastructure fails. A common enterprise SQL Server technology is **Always On Availability Groups**.

✦ 20. Always On Availability Groups

An **Always On Availability Group (AG)** maintains copies of one or more databases across multiple SQL Server instances called availability replicas.

There is one Primary Replica and one or more Secondary Replicas. The primary sends transaction-log records to secondaries, which harden and apply them to their secondary database copies. Traditional Availability Groups support one primary and up to eight secondary replicas.

Basic architecture



Primary replica

The **primary replica** hosts databases that normally accept `READ + WRITE` operations. Your Web API normally writes Gifts to the primary `FundraisingDb`.

Secondary replica

A **secondary replica** maintains another copy of each availability database. Depending on configuration, it can be used for failover, read-only workloads, and some backup operations.

Availability Group contains databases

EXAMPLE / FLOW

```
FundraisingAG
  +-- FundraisingDb
  +-- AnotherBusinessDb
```

Databases participating in the same Availability Group can fail over together.

AG requirement - FULL recovery

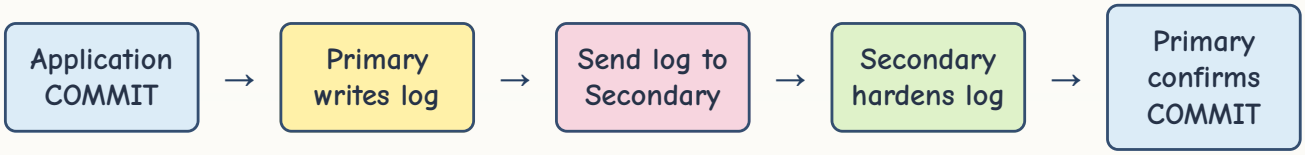
Databases participating in an Availability Group must use FULL recovery and need the appropriate backup initialization/log chain before joining.

✦ 21. Availability Modes

Two important modes are **Synchronous Commit** and **Asynchronous Commit**.

Synchronous Commit

With **synchronous commit**, the primary waits for a synchronous secondary to harden the log before confirming the protected transaction commit.



Advantages: lower risk of data loss for synchronized replicas and support for automatic/planned no-data-loss failover scenarios.

Disadvantages: higher commit latency, strong dependence on network quality, and typically best fit when replicas have low-latency connectivity.

Asynchronous Commit

With **asynchronous commit**, the primary does not wait for the secondary to harden every transaction before confirming success to the client.

EXAMPLE / FLOW

```
Application COMMIT
      |
      v
Primary writes log -----> Client receives success
      |
      v
Secondary catches up asynchronously
```

Advantages: lower primary transaction latency and useful for distant DR replicas.

Disadvantage: if the primary fails before recent log records reach/harden on the secondary, some recent transactions can be lost during forced failover.

✦ 22. Availability Group Listener

An **Availability Group Listener** provides a stable connection name for applications.

Without a listener:

EXAMPLE / FLOW

```
Web API -> SQLServer01
```

After failover to SQLServer02, the application may need a configuration change.

With a listener:

EXAMPLE / FLOW

```
Web API -> FundraisingListener -> current primary
```

A stable connection string such as `Server=FundraisingListener` lets clients connect to the appropriate primary or read-only secondary according to configuration.

✦ 23. What Is Failover?

Failover changes a secondary replica into the new primary when the current primary can no longer serve the workload or when administrators intentionally switch roles.

EXAMPLE / FLOW

```
Before: SQL01 PRIMARY   -> SQL02 SECONDARY
After:  SQL01 SECONDARY <- SQL02 PRIMARY
```

Types of Availability Group failover

- Automatic Failover
- Planned Manual Failover
- Forced Manual Failover

Automatic failover

SQL Server automatically changes the role to a configured synchronized secondary after qualifying health failure conditions. Typical requirements include synchronous commit, automatic failover configuration, and a synchronized secondary.

Planned manual failover

Administrators intentionally switch roles, for example to patch SQLServer01. They can fail over to SQLServer02, maintain SQLServer01, and later switch back. With synchronized synchronous replicas, planned manual failover can occur without data loss.

Forced failover

Used when the primary is unavailable, the secondary is not fully synchronized, and the business chooses availability over guaranteed zero data loss. Service may be restored, but data loss is possible.

Failover summary

Failover	Typical Data Loss
Automatic, synchronized	None expected
Planned manual, synchronized	None expected
Forced failover	Possible

Important AG limitation - instance-level dependencies

An Availability Group protects database availability but does not automatically synchronize every SQL Server instance-level dependency. SQL logins, SQL Agent jobs, linked server definitions, and some other instance-level objects may need separate synchronization and management.

Important database-level health note

Some database-level problems, such as a database becoming suspect because of file loss or log corruption, do not necessarily trigger failover by default unless appropriate database-level health detection is configured.

AG ≠ corruption protection.
You still need DBCC CHECKDB, backups, and restore testing.

✦ 24. Availability Group Is Not a Backup

If corruption or a harmful logical change is replicated, the secondary may receive the same problem. Therefore Always On, backup strategy, and integrity checking must coexist.

✦ 25. What Is Replication?

SQL Server Replication copies and distributes data/database objects from one database to another and synchronizes changes between them.

It is primarily designed for data distribution, reporting, distributed applications, data integration, and disconnected/mobile synchronization.

Replication is not the same as backup

Replication may copy a bad **DELETE**, bad **UPDATE**, or bad application data to subscribers. Therefore replication is not a backup strategy.

Replication is not the same as Availability Groups

EXAMPLE / FLOW

```
Availability Group -> database-level HA/DR
Replication        -> data distribution/synchronization
```

Different goals.

✦ 26. Replication Main Roles

Important concepts are Publisher, Distributor, Subscriber, Publication, Article, and Subscription.

Publisher

The **Publisher** is the server/database containing the source data being replicated.

Article

An **article** is an individual object selected for replication, such as `fundraising.Gift` or `fundraising.Campaign`.

Publication

A **publication** is a collection of replication articles, for example:

EXAMPLE / FLOW

```
FundraisingReportingPublication
  +-- Gift
  +-- GiftAllocation
  +-- Campaign
```

Distributor

The **Distributor** coordinates/stores replication information and helps move changes from the Publisher toward Subscribers.

Subscriber

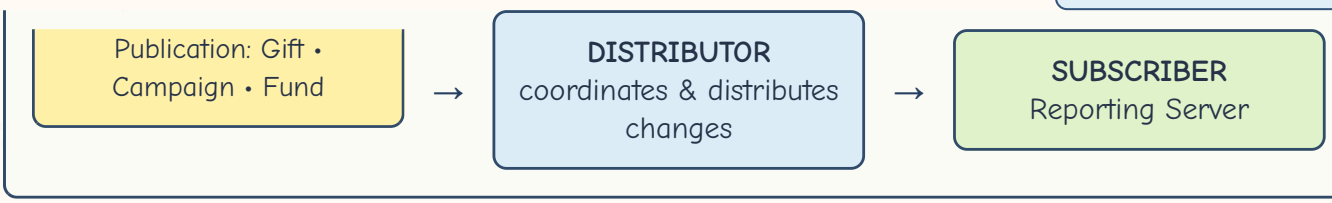
The **Subscriber** receives replicated data, for example a reporting SQL Server that receives Gift, GiftAllocation, and Campaign data.

Subscription

A **subscription** defines the relationship through which a Subscriber receives a Publication.

Replication architecture





✦ 27. Main Replication Types

SQL Server provides Snapshot Replication, Transactional Replication, and Merge Replication.

Snapshot Replication

Snapshot replication sends a complete point-in-time copy of the selected data to subscribers. It does not continuously track incremental changes after that snapshot.

EXAMPLE / FLOW

```
Publisher -> Create full snapshot -> Send snapshot -> Subscriber
```

Good when data changes infrequently, data volume is manageable, and a complete refresh is acceptable.

Transactional Replication

Transactional replication sends changes from the Publisher to Subscribers as the changes occur, typically with low latency.

EXAMPLE / FLOW

```
Initial snapshot
      |
Subscriber initialized
      |
Publisher transactions
      |
Transaction Log
      |
Log Reader Agent
      |
Distributor
      |
Distribution Agent
      |
Subscriber
```

A Gift inserted on the primary can be copied to a reporting database with low delay. Good uses include a reporting server, data-warehouse feed, server-to-server distribution, and offloading certain read workloads.

Merge Replication

Merge replication allows changes at both Publisher and Subscriber, then synchronizes those changes and resolves conflicts.

EXAMPLE / FLOW

```
Publisher <-> changes <-> Subscriber
```

Remote fundraising offices could work offline and synchronize later. If both sides changed the same data, conflict resolution is required.

Replication types comparison

Type	Main Behavior
Snapshot	Periodic complete copy
Transactional	Incremental near-real-time changes
Merge	Changes from multiple sides synchronized

Availability Group vs Replication

Availability Group	Replication
HA/DR focus	Data-distribution focus
Database replicas	Selected publications/articles
Failover capability	Publisher/subscriber model
Secondary may be readable	Subscriber serves distribution needs
Log records synchronize database replicas	Replication agents distribute selected changes

✦ 28. What Is Database Integrity Checking?

Database integrity checking verifies that the database's physical and logical structures are internally consistent. Problems can result from storage failure, hardware problems, I/O errors, corruption, or software problems.

The primary SQL Server command is `DBCC CHECKDB`.

✦ 29. DBCC CHECKDB

`DBCC CHECKDB` examines a database for physical and logical consistency problems.

SQL

```
DBCC CHECKDB('FundraisingDb');
```

It performs database allocation checks, table/view consistency checks, catalog checks, indexed-view consistency validation, and other consistency checks.

Conceptually it includes work equivalent to `DBCC CHECKALLOC`, `DBCC CHECKTABLE` for tables/views, `DBCC CHECKCATALOG`, indexed-view consistency checks, and additional database consistency checks. Because of that, you usually do not need to run all three separately when performing a full `CHECKDB`.

Typical command

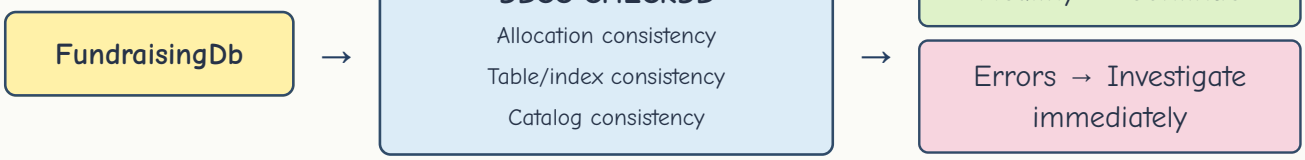
```sql
DBCC CHECKDB('FundraisingDb')
WITH NO_INFOMSGS;
```

`NO_INFOMSGS` suppresses informational messages and makes important problems easier to spot.

Why CHECKDB matters

A database can appear `ONLINE` while certain pages or structures contain corruption that normal users have not yet touched. Regular integrity checks can detect these problems before a disaster exposes them.

CHECKDB flow



CHECKDB does not replace backups

If CHECKDB says the database is healthy and somebody deletes all Gifts tomorrow, CHECKDB cannot restore deleted data.

```
EXAMPLE / FLOW
CHECKDB -> corruption detection
BACKUP  -> recovery
```

Both are required.

Backup does not replace CHECKDB

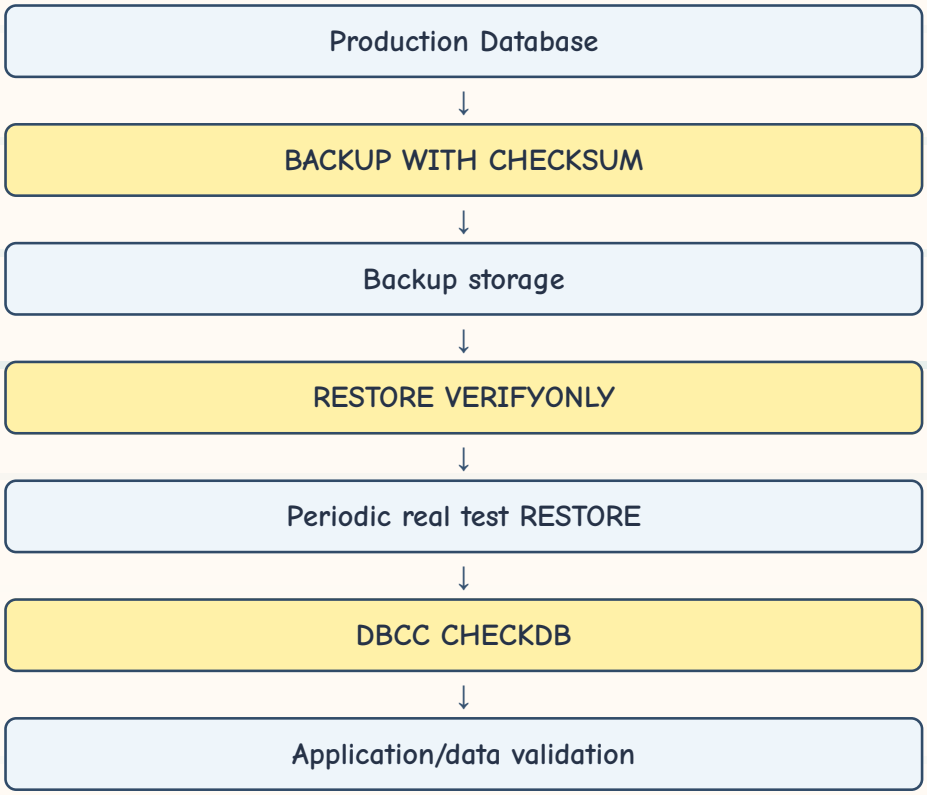
A database that already contains corruption can sometimes still be backed up, depending on the corruption and backup/checksum behavior. Backup existence alone does not prove database integrity.

✦ 30. Backup Checksums

```sql
BACKUP DATABASE FundraisingDb
TO DISK = 'D:\SQLBackups\FundraisingDb.bak'
WITH CHECKSUM;
```

Checksums help SQL Server detect certain problems while processing the backup. They are useful, but still not the same as a successful restore plus `DBCC CHECKDB`.

✦ 31. Integrity Checking and Restore Testing Together



This tests much more of the real disaster-recovery chain.

✦ 32. Maintenance Planning

A SQL Server maintenance strategy commonly includes backups, backup monitoring, restore testing, DBCC CHECKDB, statistics maintenance, targeted index maintenance, job monitoring, disk/capacity monitoring, transaction-log monitoring, Query Store monitoring, and HA health monitoring.

SQL Server Maintenance Plans can automate tasks such as Backup Database, Check Database Integrity, Reorganize/Rebuild Indexes, and Update Statistics. The built-in integrity task executes **DBCC CHECKDB**.

Should you blindly use the Maintenance Plan Wizard?

No. The schedule should be based on database size, workload, maintenance window, RPO/RTO, backup duration, CHECKDB duration, storage, and recovery requirements.

✦ 33. Example Reliability Plan for FundraisingDb

Imagine `FundraisingDb` is business-critical.

Area	Example learning plan
Recovery model	FULL
Full backup	Sunday
Differential	Daily
Log backup	Every 10 minutes
CHECKDB	Regularly, based on DB size and maintenance window
Restore test	Regularly to a separate environment
Backup storage	Separate from production storage
Off-site copy	Yes
High availability	Availability Group
DR replica	Potential asynchronous secondary in another location
Monitoring	Backup failures, AG health, disk space, CHECKDB failures, SQL Agent jobs

This is an architecture example, not a universal production schedule.

✦ 34. Backup Strategy vs HA Strategy

Think in layers:

Layer 1 - BACKUPS	Recover historical data
Layer 2 - AVAILABILITY GROUP	Reduce downtime
Layer 3 - OFF-SITE / DR	Survive site disaster
Layer 4 - DBCC CHECKDB	Detect corruption
Layer 5 - MONITORING	Detect failures quickly

No single layer replaces all the others.

✦ 35. Failure Scenarios

Accidental DELETE

```sql
DELETE
FROM fundraising.Gift;
```

Availability Group is not enough because the DELETE can replicate. Replication is not enough because the DELETE may replicate. The correct recovery tool is **backup + log backups + point-in-time recovery**.

Primary SQL Server dies

EXAMPLE / FLOW

```
SQLServer01 unavailable
```

What helps: **Availability Group + Failover**.

Entire site lost

What helps: **Remote DR copy + Off-site backups + Possibly asynchronous AG secondary + Documented recovery procedure**.

Corruption detected

What helps identify it: **DBCC CHECKDB** and page/backup checksums. What helps recover: a known-good backup and an appropriate restore strategy.

Need a reporting copy

Requirement: send selected Gift/Campaign data to another reporting environment. Potential technology: **Transactional Replication**, depending on architecture.

✦ 36. Common Backup Mistakes

Mistake 1 - Taking only full backups for a mission-critical database

If a full backup runs once each night, RPO may be close to 24 hours unless additional protection exists.

Mistake 2 - FULL recovery without log backups

This can cause transaction-log growth and gives no useful frequent recovery points. Schedule log backups when using FULL recovery.

Mistake 3 - Keeping backups on the same physical storage

One storage failure can destroy both database and backup. Keep copies separately and off-site.

Mistake 4 - Never testing restore

A successful backup does not prove disaster recovery will succeed. Perform restore tests.

Mistake 5 - Assuming VERIFYONLY is a full restore test

It verifies backup readability/completeness but does not fully validate the restored database's internal data structure.

Mistake 6 - Breaking the log chain

Under FULL recovery, preserving the required sequence of transaction-log backups is critical for recovery.

Mistake 7 - No backup monitoring

A failed backup job that nobody notices is almost the same as having no backup strategy.

✦ 37. Common Availability Mistakes

Mistake 1 - Thinking AG is backup

It is not.

Mistake 2 - Forgetting instance-level objects

Failing over databases does not automatically guarantee that Logins, SQL Agent jobs, and other instance configuration are identical on every replica.

Mistake 3 - Assuming automatic failover works with any replica

Automatic failover requires the correct synchronous commit mode, automatic failover mode, and synchronized state.

Mistake 4 - Assuming forced failover has no risk

Forced failover can involve data loss.

Mistake 5 - Never testing failover

Failover should be practiced in controlled maintenance/testing conditions.

✦ 38. Common Replication Mistakes

Mistake 1 - Treating replication as backup

Replication distributes data changes; it does not replace historical backup/recovery.

Mistake 2 - Using replication when you really need HA

If the requirement is, *"Automatically continue service when primary fails,"* an Availability Group may better match the need.

Mistake 3 - Using AG when you really need data distribution

If the requirement is, *"Send selected tables to a reporting server,"* replication may be more appropriate.

Mistake 4 - Ignoring replication latency

A subscriber may temporarily be behind the publisher. Do not automatically assume `Subscriber = exact real-time source` at every instant.

✦ 39. Common Integrity-Maintenance Mistakes

Mistake 1 - Never running CHECKDB

Corruption may remain undetected until important data is accessed.

Mistake 2 - Running CHECKDB but ignoring errors

A maintenance job only helps if failures trigger alerts and investigation.

Mistake 3 - Running maintenance without monitoring

Every important maintenance task should have success monitoring, failure alerts, and history.

Mistake 4 - Automatically running repair commands

If CHECKDB reports corruption, do not immediately run destructive repair options without understanding the situation and available backups. The preferred recovery path often involves investigation, a known-good backup, and a restore strategy rather than accepting data loss through repair.

✦ 40. Enterprise Reliability Decision Guide

Need historical recovery?
BACKUPS

Need recovery to exact time?
FULL + LOG backups

Need low server-failure downtime?
High Availability / AG

Need remote disaster recovery?
Remote secondary + off-site backups

Need selected data copied elsewhere?
Replication may fit

Need corruption detection?
DBCC CHECKDB

Need proof backups actually work?
Restore testing

41. Interview Questions - 25 Quick Answers

1. What is a full backup?

A full backup contains the entire database plus enough log information to recover that backup to a transactionally consistent state.

2. What is a differential backup?

A differential backup contains data changes since its most recent differential-base full backup.

3. Is a differential backup based on the previous differential?

Normally no. It is cumulative from the base full backup.

4. What is a transaction-log backup?

A log backup saves transaction-log records not already backed up in the previous log-backup chain and supports recovery under FULL/BULK_LOGGED models.

5. What is the SIMPLE recovery model?

A recovery model that automatically manages log reuse and does not provide the normal log-backup-based point-in-time recovery capability.

6. What is FULL recovery?

A recovery model supporting transaction-log backups and point-in-time recovery when the log chain is maintained correctly.

7. FULL vs SIMPLE?

EXAMPLE / FLOW

```
SIMPLE -> simpler -> no regular log backup chain -> no normal point-in-time recovery
FULL   -> log backups -> point-in-time recovery -> more administration
```

8. What is point-in-time recovery?

Restoring a database to a specific historical time, commonly just before a destructive or accidental change.

9. What does WITH NORECOVERY mean?

Restore the backup but leave the database in restoring state because more backups will be applied.

10. What does WITH RECOVERY mean?

Complete recovery and bring the restored database online.

11. What is a tail-log backup?

A final backup of the active transaction log taken, when possible, before beginning a restore sequence after failure so recent unbacked-up log records can be preserved.

12. What is Always On Availability Groups?

An HA/DR feature that maintains primary and secondary copies of one or more databases across SQL Server replicas and supports failover.

13. What is a primary replica?

The replica hosting the read-write primary copies of availability databases.

14. What is a secondary replica?

A replica maintaining synchronized/copy databases that can serve as failover targets and optionally support reads or some backup operations.

15. Synchronous vs asynchronous commit?

EXAMPLE / FLOW

```
Synchronous  -> primary coordinates with synchronous secondary before completing
protected commit workflow
Asynchronous -> primary does not wait for remote secondary to catch up before
confirming each commit
```

The first favors data protection; the second favors distance/latency tolerance.

16. What is automatic failover?

A configured synchronized secondary automatically takes the primary role after qualifying availability failure conditions.

17. What is forced failover?

An administrator forces a secondary into the primary role even when synchronization guarantees may not exist, so data loss is possible.

18. What is an AG listener?

A stable network/server name that applications use to connect to the appropriate Availability Group replica.

19. Does Availability Group replace backups?

No. AG secondaries are not backups, and regular database/log backups are still required.

20. What is SQL Server replication?

Technologies for distributing data/database objects from one database to others and synchronizing them.

21. What are the three main replication types?

Snapshot, Transactional, and Merge.

22. What is transactional replication?

A replication model that typically starts with a snapshot and then continuously distributes subsequent publisher changes to subscribers with low latency.

23. What is DBCC CHECKDB?

A SQL Server integrity command that checks logical and physical consistency across database structures.

24. Does RESTORE VERIFYONLY prove that a database will restore perfectly?

No. It validates that the backup is complete and readable and performs additional backup checks, but it does not fully validate the structure of the database data as an actual restore plus CHECKDB would.

25. What is the most important enterprise reliability rule?

Do not rely on one protection mechanism. A strong system combines backups, recovery planning, restore testing, integrity checks, high availability, disaster recovery, and monitoring.

Final Master Diagram

SQL SERVER RELIABILITY

BACKUP

Full Backup
Differential
Log Backup

→ **Recovery**
→ **Point-in-time**
→ **Restore chain**

HA

Availability Group
Listener
Primary / Secondary

→ **Failover**
→ **Automatic**
→ **Planned**
→ **Forced**

INTEGRITY

DBCC CHECKDB
Checksums
Restore tests

→ **Detect corruption**
→ **Validate recovery**

DISTRIBUTION

Replication
Snapshot
Transactional
Merge

→ **Copy selected data**

Final Easy Memory Rules

Full Backup
Entire database

Differential Backup
Changes since base full

Log Backup
Transaction-log changes

SIMPLE
No normal log-chain point-in-time recovery

FULL
Log backups + point-in-time recovery

NORECOVERY
More restore steps coming

RECOVERY
Finish restore and bring database online

STOPAT
Recover to a specific time

RPO
How much data can we lose?

RTO
How long can we be down?

Availability Group
Keep redundant database replicas

Primary
Read/write database

Secondary
Redundant replica/failover target

Synchronous Commit
Stronger data-protection relationship

Asynchronous Commit
Better distance/latency tolerance; possible data loss on forced failover

Failover
Secondary becomes primary

Replication
Copy/distribute selected data

Snapshot Replication
Complete point-in-time copy

Transactional Replication
Incremental near-real-time changes

Merge Replication
Both sides can change and synchronize

DBCC CHECKDB
Check database integrity

RESTORE VERIFYONLY
Check backup readability/completeness, not a full restore test

Most important rule:
Backups protect recoverability, Availability Groups protect availability, replication distributes data, and DBCC CHECKDB detects integrity problems. None of these technologies replaces all the others. A reliable enterprise SQL Server environment combines them according to the business's RPO, RTO, workload, and disaster-recovery requirements.